

Kaveri Maintenance Server

There are several auxiliary processes or services, that aid the smooth functioning of Kaveri application, that running on the **Maintenance Server**. From Kaveri's own image repository to log aggregators to a deployment orchestration platform to visualization dashboards to a proxy that serves and delegates requests to different services. The list of services are as follows

- **Kaveri's image Repository** (service name: kav2_repo)

An Image repository to store and serve all images that pertain to services in Kaveri is provisioned through this service. The service runs on port 8080 and every single docker image that is built is pushed to and pulled from this service.

This is the most critical component of the Kaveri deployment setup, and has to be treated with utmost care. Any misconfiguration here will bring the deployments to a standstill.

Command to be run to bring up the service, `docker run -d -v /DATA/docker_vol/:/var/lib/registry -p 8080:5000 --restart=always --name kav2_repo registry:2`.

The images are stored locally on a mount point `/DATA/docker_vol` on the server. This volume can and has to be pruned periodically, using the below registry maintenance commands:

- `docker exec kav2_repo bin/registry garbage-collect --dry-run /etc/docker/registry/config.yml`

This runs a **preview** of garbage collection inside the registry container. It scans all manifests and blobs, identifies what's eligible for deletion (unreferenced layers), but **does not delete anything**. Output shows how much space would be freed without making changes.

- `docker exec kav2_repo bin/registry garbage-collect --delete-untagged=true /etc/docker/registry/config.yml`

This executes **real garbage collection**. The `--delete-untagged=true` flag also removes untagged manifests (images without tags), which standard GC might preserve. It permanently deletes unreferenced blobs/layers, freeing actual disk space. To-Do - Enforce user authentication to prevent unauthorized access or modifications.

- **Prometheus**

Prometheus is a high performance data collector, scrapes metrics from configured targets i.e. servers, applications with alerting capabilities.

- **App instance** (service name: PromApp)

This instance in particular caters only to the monitoring of all critical VMs/ servers and talks to the `node_exporter` service on all configured servers defined in the configuration file which is maintained in the `Misc` Azure Repo in the following path

SNMP\AppData\prometheusApp.yml .

Command to be run to bring up the service, `docker run --restart=unless-stopped -d -p 9090:9090 -v /DATA/prom_app_vol:/prometheus --name PromApp IMG_REG/prometheus_app --config.file=/etc/prometheus/prometheus.yml --storage.tsdb.retention.time=60d --storage.tsdb.path=/prometheus`

- **DB instance** (service name: PromDb)

The DB variant is specialized to scrape metrics from Postgres databases in particular and primarily speaks to pg_exporter service on all configured postgres instances defined in the configuration file which is maintained in the Misc Azure Repo in the following path SNMP\AppData\prometheusDB.yml .

Command to be run to bring up the service, `docker run --restart=unless-stopped -d -p 9100:9090 -v /DATA/prom_db_vol:/prometheus --name PromDb IMG_REG/prometheus_db --config.file=/etc/prometheus/prometheus.yml --storage.tsdb.retention.time=60d --storage.tsdb.path=/prometheus`

- **Log aggregation**

Loki is a log aggregation system designed to store and query logs from all applications. This setup is a sender and receiver mechanism, a sender (promtail) which parses and pushes the logs to the receiver (Loki) which ingests and stores the log for future use.

- Promtail (service name: PromTail)

This service primarily served by a promtail agent which runs on the target machine itself, parsing the logs by labels as per the filter conditions from configured targets and log sinks as per the conf. This service is configured to speak to the docker service directly to pull all the logs being redirected, parse it and push to the loki instance. For this setup to work the log format has to be understood by the promtail agent and some labels will also have to be configured at the target service level.

Command to be run to bring up the service, `docker run -d --restart=unless-stopped -v /DATA/promtail:/promtail -v /var/run/docker.sock:/var/run/docker.sock --name PromTail IMG_REG/promtail .`

- Loki (service name: Loki)

Loki, the receiver instance exposes an API for promtail agents to push logs over HTTP, like so `http://loki_svc:3100/loki/api/v1/push` . The logs, so pushed are ingested and stored in a indexed manner which optimizes this data pull at any given point in time. This data is then flushed to persistent storage at a configured rate to

`/DATA/loki-data`

Command to be run to bring up the service, `docker run -d --restart=unless-stopped -v /DATA/loki-data:/loki --user "$(id -u)" --name=Loki -p 3100:3100 IMG_REG/loki .`

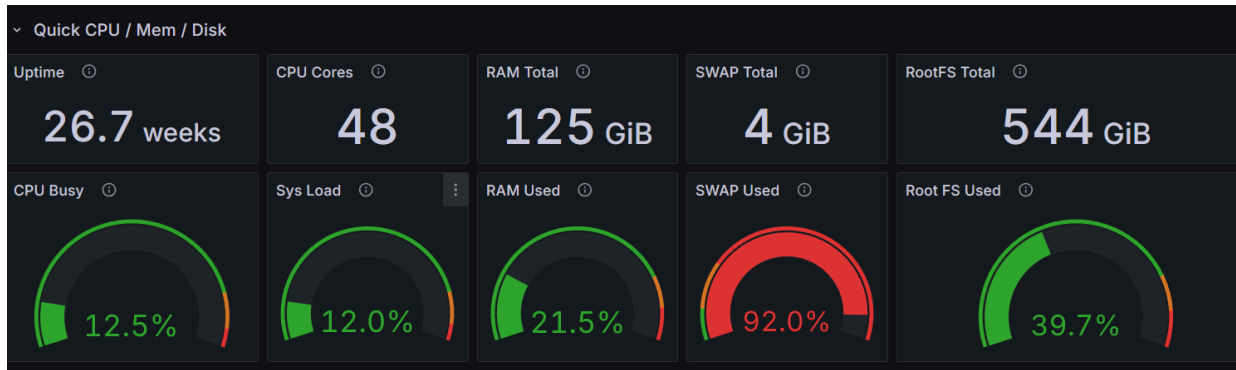
- **Grafana** (service name: grafana)

Grafana is an opensource analytics and monitoring platform that helps in visualizing and monitoring almost anything. This aids the developers/ maintainers in troubleshoot the issues faster with no cost associated as it is a self hosted instance on premise.

In Kaveri, Grafana is employed to visualize the system utilization metrics, application configuration and performance metrics over a clean intuitive dashboard. by configuring multiple data sources, we have the following available to us

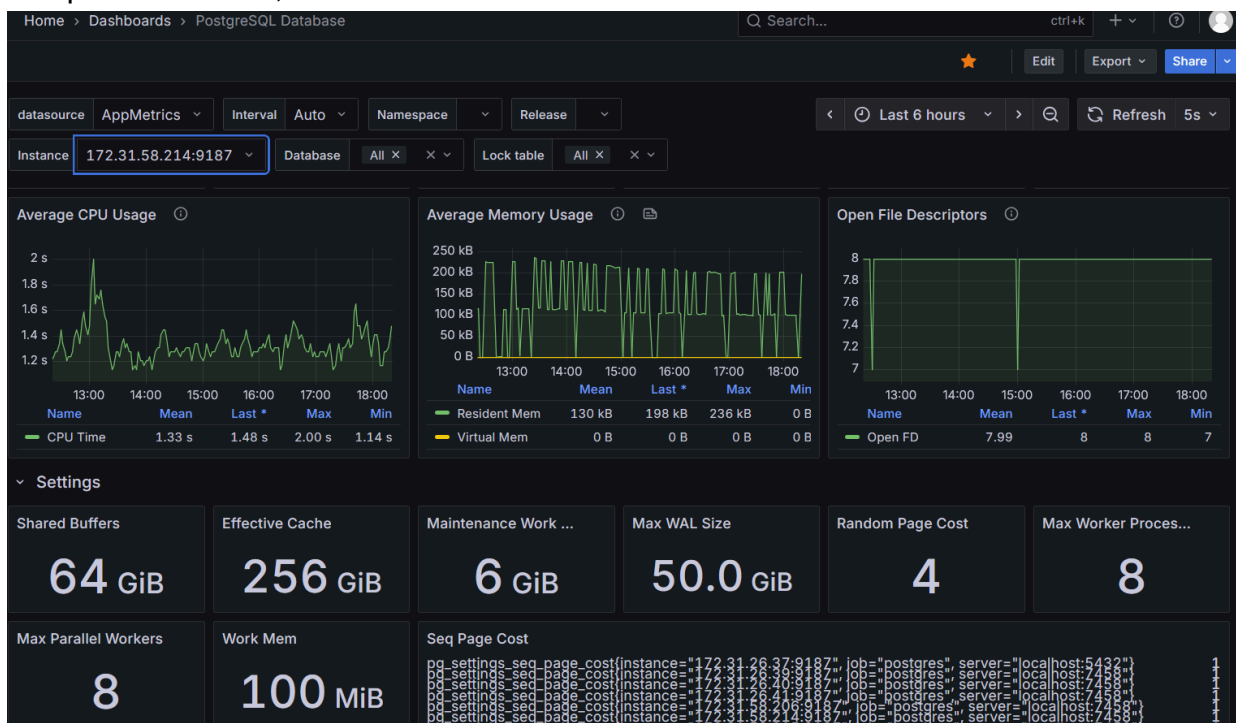
- **Kaveri App Servers** - Dashboard to monitor system and utilization metrics like uptime, system load, RAM utilization, network traffic etc.

`node_exporter` + `prometheus` setup is supplied as a data source for this. Below is a sample screenshot,



- **Postgres SQL Database** - A Dashboard to monitor the postgres instances running. This gives us an overview of the resource utilization by the process itself, transactions/ commits, locks on tables, conflicts/ deadlocks etc. along with major configuration parameters.

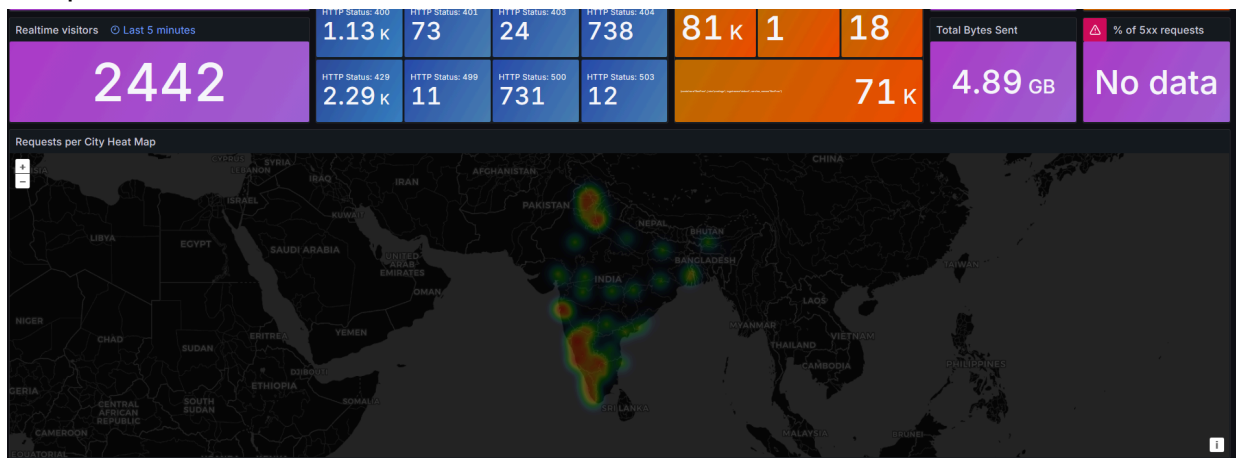
`pg_exporter` + `prometheus` setup is supplied as a data source for this. Below is a sample screenshot,



- **Grafana Loki Dashboard** - This dashboard can be a meaningful tool to understand the holistic performance of the application providing indicators like requests per status code, real-time visitor count, application traffic heatmap and other such request statistics over time. This dashboard coupled with the above dashboards can be a powerful tool to understand an issue and narrowing down to the root cause.

loki + promtail setup is supplied as a data source for this.

This dashboard also has alert manager configured, so mail notifications are triggered to concerned point of contacts for relevant limits/ thresholds crossed. Below is a sample screenshot,



Command to be run to bring up the service, `docker run --restart=unless-stopped -d -p 3000:3000 -e "GF_SECURITY_ADMIN_PASSWORD=dsr@123" --name=grafana --user "$(id -u)" -v /DATA/grafana_vol:/var/lib/grafana IMG_REG/grafana`

- **Portainer (service name: portainer)**

Portainer is an open-source tool hosted on the maintenance server itself to ease the orchestration of deployments through a graphical user interface. When coupled with a cluster management tool like `docker swarm`, this can really ease the management of deployments over the network without ever logging into the servers, all at the simple click of a button. This tool also provides mechanisms to store and manage environments, secrets, log monitoring for all stacks and services. The staging instance is managed by this tool.

To-Do - Migrate the tool for the ease production deployments.

Command to be run to bring up the service, `docker run -d -p 9000:9000 --name=portainer --restart=unless-stopped -v /var/run/docker.sock:/var/run/docker.sock -v /DATA/port_vol:/data IMG_REG/portainer`

- **Reverse Proxy (service name: prox)**

This is just a simple nginx reverse proxy serving some applications and redirecting/ routing traffic for the same services being run for managing Kaveri application. The services being run routed through this instance are as follows

- Grafana on sub-path `/grafana`, so `http://172.18.35.155/grafana` brings up the grafana interface.

- Portainer on sub-path / , so `http://172.18.35.155/` brings up the Portainer interface.

Command to be run to bring up the service, `docker run --restart=unless-stopped -d -p 80:80 --name=Prox IMG_REG/maintenance_prox`

Below is a sample `docker-compose.yml` for all the services running on the maintenance server.

`IMG_REG=172.18.35.155:8080`

```
version: "3.9"

services:
  promapp:
    container_name: PromApp
    image: IMG_REG:8080/prometheus_app
    restart: unless-stopped
    ports:
      - "9090:9090"
    volumes:
      - /DATA/prom_app_vol:/prometheus
    command:
      - "--config.file=/etc/prometheus/prometheus.yml"
      - "--storage.tsdb.retention.time=60d"
      - "--storage.tsdb.path=/prometheus"

  promdb:
    container_name: PromDb
    image: IMG_REG:8080/prometheus_db
    restart: unless-stopped
    ports:
      - "9100:9090"
    volumes:
      - /DATA/prom_db_vol:/prometheus
    command:
      - "--config.file=/etc/prometheus/prometheus.yml"
      - "--storage.tsdb.retention.time=60d"
      - "--storage.tsdb.path=/prometheus"

  prox:
    container_name: Prox
    image: IMG_REG:8080/maintenance_prox
    restart: unless-stopped
    ports:
      - "80:80"
```

```
grafana:
  container_name: grafana
  image: IMG_REG:8080/grafana
  restart: unless-stopped
  ports:
    - "3000:3000"
  environment:
    GF_SECURITY_ADMIN_PASSWORD: "dsr@123"
  # user: "${UID:-1000}"
  volumes:
    - /DATA/grafana_vol:/var/lib/grafana

nexus:
  container_name: nexus
  image: IMG_REG:8080/sonatype-nexus3
  restart: unless-stopped
  ports:
    - "8081:8081"
  volumes:
    - /DATA/nexus-data:/nexus-data

portainer:
  container_name: portainer
  image: IMG_REG:8080/portainer
  restart: unless-stopped
  ports:
    - "9000:9000"
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock
    - /DATA/port_vol:/data

loki:
  container_name: Loki
  image: IMG_REG/loki
  restart: unless-stopped
  ports:
    - "3100:3100"
  volumes:
    - /DATA/loki-data:/loki
  # user: "${UID:-1000}"

promtail:
  container_name: PromTail
  image: IMG_REG/promtail
  restart: unless-stopped
  volumes:
```

- /DATA/promtail:/promtail
- /var/run/docker.sock:/var/run/docker.sock